

Universal Windows Platform documentation

Note UWP is still supported, but the preferred way to develop Windows apps is with WinUI and the Windows App SDK.

New to UWP?



GET STARTED

[Learn the basics of UWP development](#)

[Start coding a UWP app](#)



CONCEPT

[UWP vs other Windows app platforms](#)



TRAINING

[Browse Windows courses on Microsoft Learn](#)

[What's a UWP app?](#)

Latest features



WHAT'S NEW

[Learn about the newest UWP features](#)

[Explore highlighted features of Windows 10](#)



CONCEPT

[Check out the latest in the C# language](#)

[Explore new features in Visual Studio](#)

Develop

OVERVIEW

[Build UWP apps for Windows 10](#)

[Build UWP games for Xbox and Windows 10](#)

REFERENCE

[Browse API reference docs for UWP](#)

[Explore Windows code samples](#)

DOWNLOAD

[Get Windows development tools](#)

Design

OVERVIEW

[Design your apps with the Fluent Design System](#)

REFERENCE

[Add UI controls with the Windows UI Library](#)

Publish

OVERVIEW

[Publish your app to the Microsoft Store](#)

[Package and deploy your app with MSIX](#)

CONCEPT

[Monetize your app](#)

Community and support

 OVERVIEW

[@WindowsDocs on Twitter](#) 

[OneDevMinute on YouTube](#)

[Windows developer support](#)

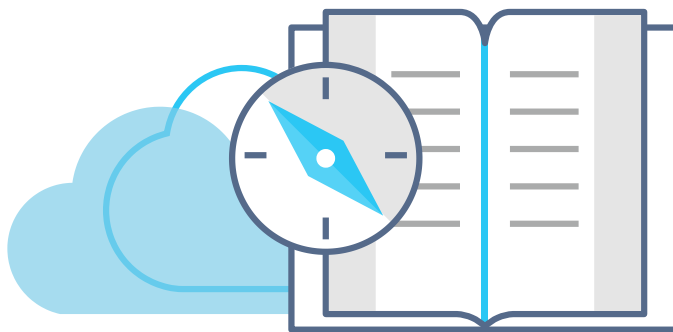
Universal Windows Platform (UWP) - Get Started

Article • 06/11/2024

ⓘ Note

The preferred approach for writing Windows apps is to use the Windows App SDK and WinUI. Another option is WPF.

Build apps that work across different Windows devices.

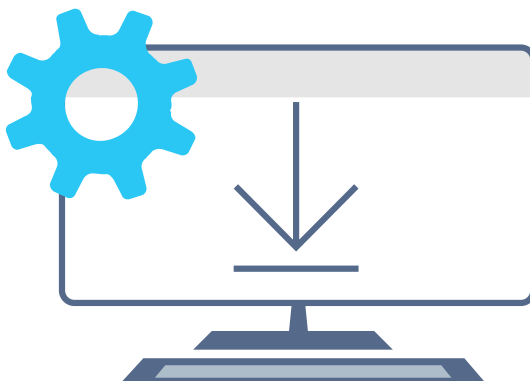


Learn about UWP

The Universal Windows Platform let you build apps that work and look great across all Windows device types, or update existing apps with new features.

[What's a UWP app?](#)

[Design and UI](#)



Get set up

Visual Studio Community and the Windows SDK give you everything you need to build great apps – and they're free.

[Download the tools and SDK](#)



Start coding

Learn how to create a UWP app.

[Start coding](#)

[API Reference](#)



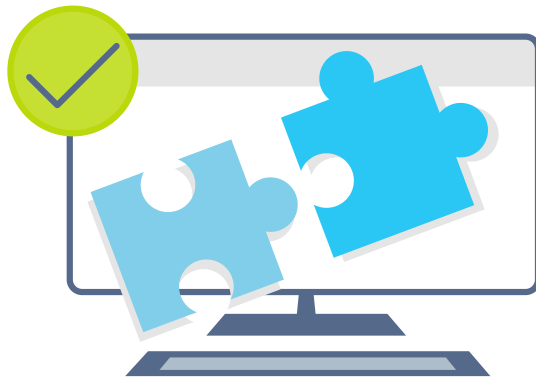
Tutorials

Looking for a guided learning experience? These tutorials will help.

[Navigate between two pages](#)

[Data binding](#)

[Use layout panels](#)



Samples

Get firsthand experience with the UWP platform, and see APIs in action.

[Using Windows app samples](#)

[Customer Orders Database](#)

[Quiz Game \(Project Rome\)](#)

[Windows code samples portal](#)



New to development?

Learn about the different options for developing Windows apps.

[Selecting a development technology](#)

[Windows on Microsoft Learn training](#)

Feedback

Was this page helpful?

☐ Yes

☐ No

What's a Universal Windows Platform (UWP) app?

Article • 08/21/2024

UWP is one of many ways to create client applications for Windows.

ⓘ Note

If you are starting to develop Windows apps, we recommend you consider using the [Windows App SDK](#), and [WinUI](#) rather than UWP. Although still supported, UWP is not under active development. Please see [Start developing Windows apps](#) for more information.

To download the tools you will need to start creating Windows apps, see [Install tools for the Windows App SDK](#), and then [write your first app](#).

Where does UWP fit in the Microsoft development story?

UWP is one choice for creating apps that run on Windows 10 and Windows 11 devices, and can be combined with other platforms. UWP apps can make use of Win32 APIs and .NET classes (see [API Sets for UWP apps](#), [DLLs for UWP apps](#), and [.NET for UWP apps](#)).

Features of a UWP app

A UWP app is:

- Secure: UWP apps declare which device resources and data they access. The user must authorize that access.
- Able to use a common API on all devices that run Windows.
- Able to use device specific capabilities and adapt the UI to different device screen sizes, resolutions, and DPI.
- Available from the Microsoft Store on all devices (or only those that you specify) that run on Windows 10 or Windows 11. The Microsoft Store provides multiple ways to make money on your app.
- Able to be installed and uninstalled without risk to the machine or incurring "machine rot".

- Engaging: use live tiles, push notifications, and user activities that interact with Windows Timeline and Cortana's Pick Up Where I Left Off, to engage users.
- Programmable in C#, C++, Visual Basic, and JavaScript. For UI, use WinUI, XAML, HTML, or DirectX.

Let's look at these in more detail.

Secure

UWP apps declare in their manifest the device capabilities they need such as access to the microphone, location, Webcam, USB devices, files, and so on. The user must acknowledge and authorize that access before the app is granted the capability.

A common API surface across all devices

Windows 10 introduced the Universal Windows Platform (UWP), which provides a common app platform on every device that runs Windows. The UWP core APIs are the same on all Windows devices. If your app only uses the core APIs, it will run on any Windows device no matter whether you are targeting a desktop PC, Xbox, Mixed-reality headset, and so on.

A UWP app written in [C++/WinRT](#) has access to the Win32 APIs that are part of the UWP. These Win32 APIs are implemented by all Windows devices.

Extension SDKs expose the unique capabilities of specific device types

If you target the universal APIs, then your app can run on all devices that run Windows 10 or later. But if you want your UWP app to take advantage of device-specific APIs, then you can do that, too.

Extension SDKs let you call specialized APIs for different devices. For example, if your UWP app targets an IoT device, you can add the IoT extension SDK to your project to target features specific to IoT devices. For more information about adding extension SDKs, see the **Extension SDKs** section in [Programming with extension SDKs](#).

You can write your app so that you expect it to run only on a particular type of device, and then limit its distribution from the Microsoft Store to just that type of device. Or, you can conditionally test for the presence of an API at runtime and adapt your app's behavior accordingly. For more information, see the **Writing code** section in [Programming with extension SDKs](#).

Adaptive controls and input

UI elements respond to the size and DPI of the screen the app is running on by adjusting their layout and scale. UWP apps work well with multiple types of input such as keyboard, mouse, touch, pen, and game controllers. If you need to further tailor your UI to a specific screen size or device, new layout panels and tooling help you design UI that can adapt to the different devices and form factors that your app may run on.



Windows helps you target your UI to multiple devices with the following features:

- Universal controls and layout panels help you to optimize your UI for the screen resolution of the device. For example, controls such as buttons and sliders automatically adapt to device screen size and DPI density. Layout panels help adjust the layout of content based on the size of the screen. Adaptive scaling adjusts to resolution and DPI differences across devices.
- Common input handling allows you to receive input through touch, a pen, a mouse, a keyboard, or a game controller.
- Tooling that helps you to design UI that can adapt to different screen resolutions.

Some aspects of your app's UI will automatically adapt across devices. Your app's user-experience design, however, may need to adapt depending on the device the app is running on. For example, a photo app could adapt its UI when running on a small, handheld device to ensure that usage is ideal for single-handed use. When a photo app is running on a desktop computer, the UI should adapt to take advantage of the additional screen space.

There's one store for all devices

A unified app store makes your app available on Windows devices such as PC, tablet, Xbox, HoloLens, Surface Hub, and Internet of Things (IoT) devices. You can submit your app to the store and make it available to all types of devices, or only those you choose. You submit and manage all your apps for Windows devices in one place. Have a C++ desktop app that you want to modernize with UWP features and sell in the Microsoft store? That's okay, too.

UWP apps integrate with [Application Insights](#) for detailed telemetry and analytics—a crucial tool for understanding your users and improving your apps.

UWP apps can be packaged with [MSIX](#) and distributed via the Microsoft Store, or in other ways. MSIX allows apps to be updated no matter how they are distributed, see [Update non-Store published app packages from your code](#).

Monetize your app

You can choose how you'll monetize your app. There are a number of ways to make money with your app. All you need to do is choose the one that works best for you, for example:

- A paid download is the simplest option. Just name the price.
- Trials let users try your app before buying it, providing easier discoverability and conversion than the more traditional "freemium" options.
- Sale prices to incentivize users.
- In-app purchases.

Deliver relevant, real-time info to your users to keep them coming back

There are a variety of ways to keep users engaged with your UWP app:

- Live tiles and lock screen tiles that show contextually relevant and timely info from your app at a glance.
- Push notifications that bring real-time alerts to your user's attention.
- User Activities allow users to pick up where they left off in your app, even across devices.
- The Action Center organizes notifications from your app.
- Background execution and triggers bring your app into action when the user needs it.
- Your app can use voice and Bluetooth LE devices to help users interact with the world around them.
- Integrate Cortana to add voice command capability to your app.

Use a language you already know

UWP apps use the Windows Runtime, the native API provided by the operating system. This API is implemented in C++ and is supported in C#, Visual Basic, C++, and JavaScript. Some options for writing UWP apps include:

- XAML UI and C#, VB, or C++
- DirectX UI and C++
- JavaScript and HTML
- WinUI

Links to help you get going

Get set up

Check out [Get set up](#) to download the tools you need to start creating apps, and then [write your first app](#).

Design your app

The Microsoft design system is named Fluent. The Fluent Design System is a set of UWP features combined with best practices for creating apps that perform beautifully on all types of Windows-powered devices. Fluent experiences adapt and feel natural on devices from tablets to laptops, from PCs to televisions, and on virtual reality devices. See [The Fluent Design System for UWP apps](#) for an introduction to Fluent Design.

Good [design](#) is the process of deciding how users will interact with your app, in addition to how it will look and function. User experience plays a huge part in determining how happy people will be with your app, so don't skimp on this step. [Design basics](#) introduces you to designing a Universal Windows app. See the [device primer](#) to help you think through the interaction experience of using your app on all the different form factors you want to target.

In addition to interaction on different devices, [plan your app](#) to embrace the benefits of working across multiple devices. For example:

- Design your workflow using [Navigation design basics for UWP apps](#) to accommodate mobile, small-screen, and large-screen devices. [Lay out your user interface](#) to respond to different screen sizes and resolutions.
- Consider how you'll accommodate multiple kinds of input. See the [Guidelines for interactions](#) to learn how users can interact with your app by using [Speech](#), [Touch interactions](#), the [Touch keyboard](#) and more. Or, see the [Guidelines for text and text input](#) for more traditional interaction experiences.

Add services

- Use [Azure Cloud Services](#) to sync across devices.
- Learn how to [connect to web services](#) to support your app experience.
- Include [Push notifications](#) and [in-app purchases](#) in your planning. These features should work across devices.

Submit your app to the Store

[Partner Center](#)  lets you manage and submit all of your apps for Windows devices in one place. See [Publish Windows apps and games](#) to learn how to submit your apps for publication in the Microsoft Store.

New features simplify processes while giving you more control. You'll also find detailed [analytic reports](#) combined [payout details](#), ways to [promote your app and engage with your customers](#), and much more.

For more introductory material, see [An Introduction to Building Windows Apps for Windows 10 Devices](#)

More advanced topics

- Learn how to use [User Activities](#)  so that user activity in your app appear in Windows Timeline and Cortana's Pick Up Where I Left Off feature.
- Learn how to use [Tiles, badges, and notifications for UWP apps](#).
- For the full list of Win32 APIs available to UWP apps, see [API Sets for UWP apps](#) and [DLLs for UWP apps](#).
- See [Universal Windows apps in .NET](#)  for an overview of writing .NET UWP apps.
- For a list of .NET types that you can use in a UWP app, see [.NET for UWP apps](#)
- [Compiling apps with .NET Native](#)
- Learn how to add modern experiences for Windows users to your existing desktop app, and distribute it in the Microsoft Store with the [Desktop Bridge](#).

How the Universal Windows Platform relates to Windows Runtime APIs

If you're building a Universal Windows Platform (UWP) app, then you can get a lot of mileage and convenience out of treating the terms "Universal Windows Platform (UWP)" and "Windows Runtime (WinRT)" as more or less synonymous. But it is possible to look under the covers of the technology, and determine just what the difference is between those ideas. If you're curious about that, then this last section is for you.

The Windows Runtime, and WinRT APIs, are an evolution of Windows APIs. Originally, Windows was programmed via flat, C-style Win32 APIs. To those were added COM APIs ([DirectX](#) being a prominent example). Windows Forms, WPF, .NET, and managed languages brought their own way of writing Windows apps, and their own flavor of API technology. The Windows Runtime is, under the covers, the next stage of COM. At the actual application binary interface (ABI) layer, its roots in COM become visible. But the Windows Runtime was designed to be callable from a great range of different programming languages. And callable in a way that's very natural to each of those languages. To this end, access to the Windows Runtime is made available via what are known as language projections. There is a Windows Runtime language projection into C#, into Visual Basic, into standard C++, into JavaScript, and so on. Furthermore, once packaged appropriately (see [Desktop Bridge](#)), you can call WinRT APIs from an app built in one of a great range of application models: Win32, .NET, WinForms, and WPF.

And, of course, you can call WinRT APIs from your UWP app. UWP is an application model built on top of the Windows Runtime. Technically, the UWP application model is based on [CoreApplication](#), although that detail may be hidden from you, depending on your choice of programming language. As this topic has explained, from a value proposition point of view, the UWP lends itself to writing a single binary that can, should you choose, be published to the Microsoft Store and run on any one of a great range of device form factors. The device reach of your UWP app depends on the subset of Windows Runtime APIs that you limit your app to calling, or that you call conditionally.

Hopefully, this section has been successful in describing the difference between the technology underlying Windows Runtime APIs, and the mechanism and business value of the Universal Windows Platform.

Feedback

Was this page helpful?

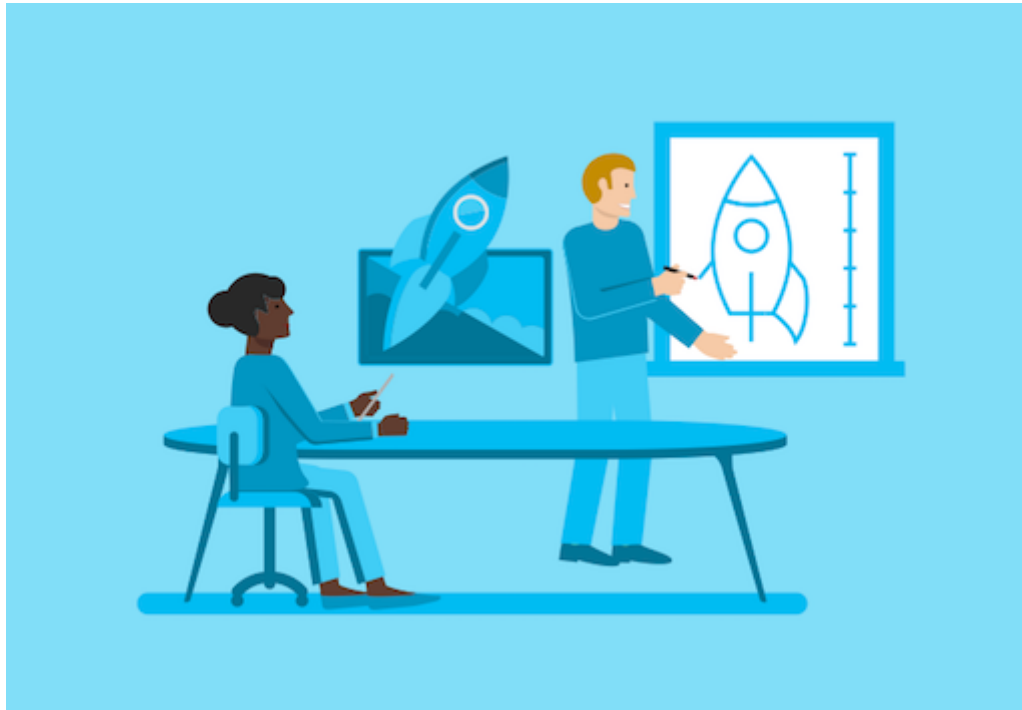
 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Start coding

Article • 08/21/2024



Welcome to the [UWP platform](#)!

ⓘ Note

If you are starting to develop Windows apps, we recommend you consider using the [Windows App SDK](#), and [WinUI](#) rather than UWP. Although still supported, UWP is not under active development. Please see [Start developing Windows apps](#) for more information.

Before you begin development, you should be sure to [get set up](#).

Learning tracks

The following learning tracks show you what you need to know to complete some basic tasks, and where you can find that info. They aren't tutorials, but they provide reference code to ensure you're on the right track.

[Expand table](#)

Task	Description
Construct a form	Learn how to create a form that's easy to use and looks good no matter the screen size.
Display customers in a list	Learn how to display and edit data in your UI.
Save and load settings	Learn how and where to save and retrieve settings.
Work with files	Learn how to read from and write to files, and which folders you do and don't have access to.

All our learning tracks are written for experienced devs who are getting familiar with Windows. If you're just beginning, start with our content for new developers.

For new developers

If you're a new dev, Microsoft has many resources to teach you the basics of how to use the code and tools that you'll need for Windows development.

- [Create a "Hello World" app](#)

If you want a more detailed walkthrough on the basics of coding, the C# language, Visual Studio, or the features of the Universal Windows Platform, you should check out the following resources:

Docs:

- [Get started with C#](#)
- [C# Quickstarts](#)
- [Get started with Visual Studio](#)
- [Microsoft Learn training resources for Windows](#)

Using the docs

If you've already explored our learning tracks, or if you're interested in something they don't cover, you should take your own tour through our documentation. Here's a quick overview of what you can expect to find in each area:

Area	Description
What's New?	Each major update of Windows sees our docs expanded with new guidance. These docs have information about the feature into and developer guidance we've added for every release, as well as lists of new APIs: What's new for developers in Windows 11 What's new for developers in Windows 10
Design and UI	All information on the visual presentation and UI of your app is contained in this area of the docs. If you're interested on the specifics of the XAML markup language or just want to create a unique look for your docs, start here: Design basics for UWP apps
Develop apps	If you want detailed information on a specific Windows feature, or are just interested in seeing what you can do with UWP development, check out this area of our docs: UWP app features . API reference for Windows apps is hosted in a related set of docs, which you can find here: Windows UWP namespaces File and XML schemas
Develop games	These docs contain information on how to develop games on Windows or Xbox. This includes setup instructions, developer programs, and instructions for programming with DirectX or Xbox features. Getting started with game development
Publish	These docs contain information on how to publish your app to the Windows Store, from app submission to pricing to promotion and customer engagement: Publish your app to the Windows Store

Other docs

Some specialized Windows platforms, like web development or Mixed Reality, have their own doc sets. If you're interested in developing an app with those features, check out their documentation:

 Expand table

Docs	Description
Microsoft Azure	Information on cloud development and Microsoft Azure can be found in the Microsoft Azure developer documentation .
Web development	Information on Microsoft Edge, WebVR, and other Windows web development features can be found in the Microsoft Edge developer documentation .
Windows Mixed Reality	Mixed reality blends real-world and virtual content into experiences where physical and digital objects coexists. Information on building apps for the

Docs	Description
	Microsoft HoloLens and other immersive headsets can be found in the Windows Mixed Reality documentation .

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Create and customize a form

Article • 10/20/2022

If you're creating an app that requires users to input a significant amount of information, chances are you'll want to create a form for them to fill out. This article will show you what you need to know in order to create a form that is useful and robust.

We'll discuss what **XAML controls** go into a form, how to best arrange them on your page, and how to optimize your form for changing screen sizes. But because a form is about the relative position of visual elements, let's first discuss page layout with XAML.

What do you need to know?

UWP does not have an explicit form control that you can add to your app and configure. Instead, you'll need to create a form by arranging a collection of UI elements on a page.

To do so, you'll need to understand **layout panels**. These are containers that hold your app's UI elements, allowing you to arrange and group them. Placing layout panels within other layout panels gives you a great deal of control over where and how your items display in relation to one another. This also makes it far easier to adapt your app to changing screen sizes.

Read [this documentation on layout panels](#). Because forms are usually displayed in one or more vertical columns, you'll want to group similar items in a **StackPanel**, and arrange those within a **RelativePanel** if you need to. Start putting together some panels now — if you need a reference, below is a basic layout framework for a two-column form:

XAML

```
<RelativePanel>
    <StackPanel x:Name="Customer" Margin="20">
        <!--Content-->
    </StackPanel>
    <StackPanel x:Name="Associate" Margin="20"
RelativePanel.RightOf="Customer">
        <!--Content-->
    </StackPanel>
    <StackPanel x:Name="Save" Orientation="Horizontal"
RelativePanel.Below="Customer">
        <!--Save and Cancel buttons-->
    </StackPanel>
</RelativePanel>
```

What goes in a form?

You'll need to fill your form with an assortment of [XAML Controls](#). You're probably familiar with those, but feel free to read up if you need a refresher. In particular, you'll want controls that allow your user to input text or choose from a list of values. This is a basic list of options you could add – you don't need to read everything about them, just enough so you understand what they look like and how they work.

- [TextBox](#) lets a user input text into your app.
- [ToggleSwitch](#) lets a user choose between two options.
- [DatePicker](#) lets a user select a date value.
- [TimePicker](#) lets a user select a time value.
- [ComboBox](#) expand to display a list of selectable items. You can learn more about them [here](#)

You also might want to add [buttons](#), so the user can save or cancel.

Format controls in your layout

You know how to arrange layout panels and have items you'd like to add, but how should they be formatted? The [forms](#) page has some specific design guidance. Read through the sections on **Types of forms** and **layout** for useful advice. We'll discuss accessibility and relative layout more shortly.

With that advice in mind, you should start adding your controls of choice into your layout, being sure they're given labels and spaced properly. As an example, here's the bare-bones outline for a single-page form using the above layout, controls, and design guidance:

XAML

```
<RelativePanel>
    <StackPanel x:Name="Customer" Margin="20">
        <TextBox x:Name="CustomerName" Header= "Customer Name"
Margin="0,24,0,0" HorizontalAlignment="Left" />
        <TextBox x:Name="Address" Header="Address" PlaceholderText="Address"
Margin="0,24,0,0" HorizontalAlignment="Left" />
        <TextBox x:Name="Address2" Margin="0,24,0,0"
PlaceholderText="Address 2" HorizontalAlignment="Left" />
        <RelativePanel>
            <TextBox x:Name="City" PlaceholderText="City"
Margin="0,24,0,0" HorizontalAlignment="Left" />
            <ComboBox x:Name="State" PlaceholderText="State"
Margin="24,24,0,0" RelativePanel.RightOf="City">
                <!--List of valid states-->
            </ComboBox>
```

```

        </RelativePanel>
    </StackPanel>
    <StackPanel x:Name="Associate" Margin="20"
RelativePanel.Below="Customer">
        <TextBox x:Name="AssociateName" Header= "Associate"
Margin="0,24,0,0" HorizontalAlignment="Left" />
        <DatePicker x:Name="TargetInstallDate" Header="Target install Date"
HorizontalAlignment="Left" Margin="0,24,0,0"></DatePicker>
        <TimePicker x:Name="InstallTime" Header="Install Time"
HorizontalAlignment="Left" Margin="0,24,0,0"></TimePicker>
    </StackPanel>
    <StackPanel x:Name="Save" Orientation="Horizontal"
RelativePanel.Below="Associate">
        <Button Content="Save" Margin="24" />
        <Button Content="Cancel" Margin="24" />
    </StackPanel>
</RelativePanel>

```

Feel free to customize your controls with more properties for a better visual experience.

Make your layout responsive

Users might view your UI on a variety of devices with different screen widths. To ensure that they have a good experience regardless of their screen, you should use [responsive design](#). Read through that page for good advice on the design philosophies to keep in mind as you proceed.

The [Responsive layouts with XAML](#) page gives a detailed overview of how to implement this. For now, we'll focus on **fluid layouts** and **visual states in XAML**.

The basic form outline that we've put together is already a **fluid layout**, as it's depending mostly on the relative position of controls with only minimal use of specific pixel sizes and positions. Keep this guidance in mind for more UIs you might create in the future, though.

More important to responsive layouts are **visual states**. A visual state defines property values that are applied to a given element when a given condition is true. [Read up on how to do this in xaml](#), and then implement them into your form. Here's what a *very* basic one might look like in our previous sample:

XAML

```

<Page ...>
    <Grid>
        <VisualStateManager.VisualStateGroups>
            <VisualStateGroup>
                <VisualState>

```

```

        <VisualState.StateTriggers>
            <AdaptiveTrigger MinWindowWidth="640" />
        </VisualState.StateTriggers>

        <VisualState.Setters>
            <Setter Target="Associate.(RelativePanel.RightOf)"
Value="Customer"/>
            <Setter Target="Associate.(RelativePanel.Below)"
Value=""/>
            <Setter Target="Save.(RelativePanel.Below)"
Value="Customer"/>
        </VisualState.Setters>
    </VisualState>
</VisualStateGroup>
</VisualStateManager.VisualStateGroups>

    <RelativePanel>
        <!-- Customer StackPanel -->
        <!-- Associate StackPanel -->
        <!-- Save StackPanel -->
    </RelativePanel>
</Grid>
</Page>

```

❗ Important

When you use StateTriggers, always ensure that VisualStateGroups is attached to the first child of the root. Here, **Grid** is the first child of the root **Page** element.

It's not practical to create visual states for a wide array of screen sizes, nor are more than a couple likely to have significant impact on the user experience of your app. We recommend designing instead for a few key breakpoints - you can [read more here](#).

Add accessibility support

Now that you have a well-constructed layout that responds to changes in screen sizes, a last thing you can do to improve the user experience is to [make your app accessible](#). There's a lot that can go into this, but in a form like this one it's easier than it looks. Focus on the following:

- Keyboard support - ensure the order of elements in your UI panels match how they're displayed on screen, so a user can easily tab through them.
- Screen reader support - ensure all your controls have a descriptive name.

When you're creating more complex layouts with more visual elements, you'll want to consult the [accessibility checklist](#) for more details. After all, while accessibility isn't

necessary for an app, it helps it reach and engage a larger audience.

Going further

Though you've created a form here, the concepts of layouts and controls are applicable across all XAML UIs you might construct. Feel free to go back through the docs we've linked you to and experiment with the form you have, adding new UI features and further refining the user experience. If you want step-by-step guidance through more detailed layout features, see our [adaptive layout tutorial](#)

Forms also don't have to exist in a vacuum - you could go one step further and embed yours within a [list/details pattern](#) or a [NavigationView](#). Or if you want to get to work on the code-behind for your form, you might want to get started with our [events overview](#).

Useful APIs and docs

Here's a quick summary of APIs and other useful documentation to help you get started working with Data Binding.

Useful APIs

API	Description
Controls useful for forms	A list of useful input controls for creating forms, and basic guidance of where to use them.
Grid	A panel for arranging elements in multi-row and multi-column layouts.
RelativePanel	A panel for arranging items in relation to other elements and to the panel's boundaries.
StackPanel	A panel for arranging elements into a single horizontal or vertical line.
VisualState	Allows you to set the appearance of UI elements when they're in particular states.

Useful docs

Topic	Description
Accessibility overview	A broad-scale overview of accessibility options in apps.

Topic	Description
Accessibility checklist	A practical checklist to ensure your app meets accessibility standards.
Events overview	Details on adding and structuring events to handle UI actions.
Forms	Overall guidance for creating forms.
Layout panels	Provides an overview of the types of layout panels and where to use them.
List/details pattern	A design pattern that can be implemented around one or multiple forms.
NavigationView	A control that can contain one or multiple forms.
Responsive design	An overview of large-scale responsive design principles.
Responsive layouts with XAML	Specific information on visual states and other implementations of responsive design.
Screen sizes for responsive design	Guidance on which screen sizes to which responsive layouts should be scoped.

Useful code samples

Code sample	Description
Customer Orders Database 	See layout and forms in action on a multi-page enterprise sample.
XAML Controls Gallery 	See a selection of XAML controls, and how they're implemented.
Additional code samples	Choose Controls , layout , and text in the category drop-down list to see related code samples.

Display customers in a list

Article • 10/20/2022

Displaying and manipulating real data in the UI is crucial to the functionality of many apps. This article will show you what you need to know to display a collection of Customer objects in a list.

This is not a tutorial. If you want one, see our [data binding tutorial](#), which will provide you with a step-by-step guided experience.

We'll start with a quick discussion of data binding - what it is and how it works. Then we'll add a **ListView** to the UI, add data binding, and customize the data binding with additional features.

What do you need to know?

Data binding is a way to display an app's data in its UI. This allows for *separation of concerns* in your app, keeping your UI separate from your other code. This creates a cleaner conceptual model that's easier to read and maintain.

Every data binding has two pieces:

- A source which provides the data to be bound.
- A target in the UI where the data is displayed.

To implement a data binding, you'll need to add code to your source that provides data to the binding. You'll also need to add one of two markup extensions to your XAML to specify the data source properties. Here's the key difference between the two:

- **x:Bind** is strongly typed, and generates code at compile time for better performance. x:Bind defaults to a one-time binding, which optimizes for the fast display of read-only data that doesn't change.
- **Binding** is weakly typed and assembled at runtime. This results in poorer performance than with x:Bind. In almost all cases, you should use x:Bind instead of Binding. However, you're likely to encounter it in older code. Binding defaults to one-way data transfer, which optimizes for read-only data that can change at the source.

We recommend you use **x:Bind** whenever possible, and we'll be showing it in the snippets in this article. For more information on the differences, see [{x:Bind} and {Binding} feature comparison](#).

Create a data source

First, you'll need a class to represent your Customer data. To give you a reference point, we'll be showing the process on this bare-bones example:

C#

```
public class Customer
{
    public string Name { get; set; }
}
```

Create a list

Before you can display any customers, you need to create the list to hold them. The [ListView](#) is a basic XAML control which is ideal for this task. Your ListView currently requires a position on the page, and will shortly need a value for its **ItemSource** property.

XAML

```
<ListView ItemsSource=""
    HorizontalAlignment="Center"
    VerticalAlignment="Center"/>
```

Once you have bound data to your ListView, we encourage you to return to the documentation, and experiment with customizing its appearance and layout to best fit your needs.

Bind data to your list

Now that you've made a basic UI to hold your bindings, you need to configure your source to provide them. Here's an example of how this may be done:

C#

```
public sealed partial class MainPage : Page
{
    public ObservableCollection<Customer> Customers { get; }
        = new ObservableCollection<Customer>();

    public MainPage()
    {
        this.InitializeComponent();
        // Add some customers
    }
}
```

```

        this.Customers.Add(new Customer() { Name = "NAME1" });
        this.Customers.Add(new Customer() { Name = "NAME2" });
        this.Customers.Add(new Customer() { Name = "NAME3" });
    }
}

```

XAML

```

<ListView ItemsSource="{x:Bind Customers}"
    HorizontalAlignment="Center"
    VerticalAlignment="Center">
    <ListView.ItemTemplate>
        <DataTemplate x:DataType="local:Customer">
            <TextBlock Text="{x:Bind Name}"/>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>

```

The [Data Binding overview](#) walks you through a similar problem, in its section about binding to a collection of items. Our example here shows the following crucial steps:

- In the code-behind of your UI, create a property of type **ObservableCollection<T>** to hold your Customer objects.
- Bind your ListView's **ItemSource** to that property.
- Provide a basic **ItemTemplate** for the ListView, which will configure how each item in the list is displayed.

Feel free to look back at the [List View](#) docs if you want to customize layout, add item selection, or tweak the **DataTemplate** you just made. But what if you want to edit your Customers?

Edit your Customers through the UI

You've displayed customers in a list, but data binding lets you do more. What if you could edit your data directly from the UI? To do this, let's first talk about the three modes of data binding:

- *One-Time*: This data binding is only activated once, and doesn't react to changes.
- *One-Way*: This data binding will update the UI with any changes made to the data source.
- *Two-Way*: This data binding will update the UI with any changes made to the data source, and also update the data with any changes made within the UI.

If you've followed the code snippets from earlier, the binding you made uses `x:Bind` and doesn't specify a mode, making it a One-Time binding. If you want to edit your

Customers directly from the UI, you'll need to change it to a Two-Way binding, so that changes from the data will be passed back to the Customer objects. [Data binding in depth](#) has more information.

Two-way binding will also update the UI if the data source is changed. For this to work, you must implement **INotifyPropertyChanged** on the source and ensure its property setters raise the **PropertyChanged** event. Common practice is to have them call a helper method like the **OnPropertyChanged** method, as shown below:

C#

```
public class Customer : INotifyPropertyChanged
{
    private string _name;
    public string Name
    {
        get => _name;
        set
        {
            if (_name != value)
            {
                _name = value;
                this.OnPropertyChanged();
            }
        }
    }
    public event PropertyChangedEventHandler PropertyChanged;

    public void OnPropertyChanged([CallerMemberName] string propertyName =
null) =>
        this.PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
}
```

Then make the text in your ListView editable by using a **TextBox** instead of a **TextBlock**, and ensure that you set the **Mode** on your data bindings to **TwoWay**.

XAML

```
<ListView ItemsSource="{x:Bind Customers}"
    HorizontalAlignment="Center"
    VerticalAlignment="Center">
    <ListView.ItemTemplate>
        <DataTemplate x:DataType="local:Customer">
            <TextBox Text="{x:Bind Name, Mode=TwoWay}"/>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

A quick way to ensure that this works is to add a second `ListView` with `TextBox` controls and `OneWay` bindings. The values in the second list will automatically change as you edit the first one.

ⓘ Note

Editing directly inside a `ListView` is a simple way to show Two-Way binding in action, but can lead to usability complications. If you're looking to take your app further, consider using **other XAML controls** to edit your data, and keep your `ListView` as display-only.

Going Further

Now that you've created a list of customers with two-way binding, feel free to go back through the docs we've linked you to and experiment. You can also check out our [data binding tutorial](#) if you want a step-by-step walkthrough of basic and advanced bindings, or investigate controls like the [list/details pattern](#) to make a more robust UI.

Useful APIs and docs

Here's a quick summary of APIs and other useful documentation to help you get started working with Data Binding.

Useful APIs

API	Description
Data template	Describes the visual structure of a data object, allowing for the display of specific elements in the UI.
x:Bind	Documentation on the recommended <code>x:Bind</code> markup extension.
Binding	Documentation on the older <code>Binding</code> markup extension.
ListView	A UI control that displays data items in a vertical stack.
TextBox	A basic text control for displaying editable text data in the UI.
INotifyPropertyChanged	The interface for making data observable, providing it to a data binding.
ItemsControl	The ItemsSource property of this class allows a <code>ListView</code> to bind to a data source.

Useful docs

Topic	Description
Data binding in depth	A basic overview of data binding principles
Data Binding overview	Detailed conceptual information on data binding.
List View	Information on creating and configuring a ListView, including implementation of a DataTemplate

Useful code samples

Code sample	Description
Data binding tutorial	A step-by-step guided experience through the basics of data binding.
ListView and GridView 	Explore more elaborate ListViews with data binding.
QuizGame 	See data binding in action, including the BindableBase class (in the "Common" folder) for a standard implementation of INotifyPropertyChanged .

Save and load settings in a UWP app

Article • 10/20/2022

This topic covers what you need to know to get started loading, and saving, settings in a Universal Windows Platform (UWP) app. The main APIs are introduced, and links are provided to help you learn more.

Use settings to remember the user-customizable aspects of your app. For example, a news reader could use app settings to save which news sources to display and what font to use for reading articles.

We'll look at code to save and load app settings.

What do you need to know

Use app settings to store configuration data such as user preferences and app state.

The following data types can be used with settings: integers, doubles, floats, chars, strings, Points, DateTimes, and more. You can also store instances of the [ApplicationDataCompositeValue](#) class which is useful when there are multiple settings that should be treated as a unit. For example, a font name and point size for displaying text in the reading pane of your app should be saved/restored as a single unit. This prevents one setting from getting out of sync with the other due to concurrent access, such as between the main program and a background task.

Here are the main APIs you need to know about to save or load app settings:

- [Windows.Storage.ApplicationData.Current.LocalSettings](#) gets the application settings container from the local app data store. Settings stored here are kept on the device.
- [Windows.Storage.ApplicationData.Current.RoamingSettings](#) gets the application settings container from the roaming app data store. Settings stored here no longer roam (as of Windows 11), but the settings store is still available. The recommended replacement for RoamingSettings is [Azure App Service](#). Azure App Service is widely supported, well documented, reliable, and supports cross-platform/cross-ecosystem scenarios such as iOS, Android and web.
- [Windows.Storage.ApplicationDataContainer](#) is a container that represents app settings as key/value pairs. Use this class to create and retrieve setting values.
- [Windows.Storage.ApplicationDataCompositeValue](#) represents multiple app settings that should be serialized as a unit. This is useful when one setting shouldn't be updated independently of another.

Save app settings

For this introduction, we will focus on two simple scenarios: saving and loading a simple app setting, and saving and loading a composite font/font size setting.

C#

```
ApplicationDataContainer localSettings =  
Windows.Storage.ApplicationData.Current.LocalSettings;  
  
// Save a setting locally on the device  
localSettings.Values["test setting"] = "a device specific setting";  
  
// Save a composite setting locally on the device  
Windows.Storage.ApplicationDataCompositeValue composite = new  
Windows.Storage.ApplicationDataCompositeValue();  
composite["Font"] = "Calibri";  
composite["FontSize"] = 11;  
localSettings.Values["FontInfo"] = composite;
```

Save a setting by first getting an **ApplicationDataContainer** for the local settings data store with `Windows.Storage.ApplicationData.Current.LocalSettings`. Key/value dictionary pairs that you assign to this instance are saved in the local device setting data store.

In the code snippet above, an **ApplicationDataCompositeValue** stores multiple key/value pairs. Composite values are useful when you have multiple settings that shouldn't get out of sync with each other. When you save a **ApplicationDataCompositeValue**, the values are saved and loaded as a unit, or atomically. This way settings that are related won't get out of sync.

Load app settings

C#

```
ApplicationDataContainer localSettings =  
Windows.Storage.ApplicationData.Current.LocalSettings;  
  
// load a setting that is local to the device  
String localValue = localSettings.Values["test setting"] as string;  
  
// load a composite setting  
Windows.Storage.ApplicationDataCompositeValue composite =  
(ApplicationDataCompositeValue)localSettings.Values["FontInfo"];  
if (composite != null)  
{  
    String fontName = composite["Font"] as string;
```



```
int fontSize = (int)composite["FontSize"];  
}
```

Load a setting by first getting an **ApplicationDataContainer** instance for the local settings data store with `Windows.Storage.ApplicationData.Current.LocalSettings`. Then use it to retrieve key/value pairs. If the data is not present, you'll get a null object. In C#, this means that the `localValue` will be `null` if the setting does not exist.

Load a composite setting by following a similar pattern. Access key/value pairs from the local settings data store. If the data is not present, you'll get a null **ApplicationDataContainer**. That's why there is a `if (composite != null)` check in the example code above.

Useful APIs and docs

Here is a quick summary of APIs, and other useful documentation, to help get you started saving and loading app settings.

Useful APIs

API	Description
ApplicationData.LocalSettings	Gets the application settings container from the local app data store.
ApplicationData.RoamingSettings	Gets the application settings container from the roaming app data store.
ApplicationDataContainer	A container for app settings that supports creating, deleting, enumerating, and traversing the container hierarchy.
Windows.UI.ApplicationSettings Namespace	Provides classes that you'll use to define the app settings that appear in the settings pane of the Windows shell.

Useful docs

Topic	Description
Guidelines for app settings	Describes best practices for creating and displaying app settings.
Store and retrieve settings and other app data	Walk-through for saving and retrieving settings.

Useful code samples

Code sample	Description
Application data sample 	Scenarios 2-4 focus on settings

Work with files

Article • 10/20/2022

This topic covers what you need to know to get started reading from, and writing to, files in a Universal Windows Platform (UWP) app. The main APIs and types are introduced, and links are provided to help you learn more.

This is not a tutorial. If you want a tutorial, see [Create, write, and read a file](#) which, in addition to demonstrating how to create, read, and write a file, shows how to use buffers and streams. You may also be interested in the [File access sample](#) [↗](#) which shows how to create, read, write, copy and delete a file, as well as how to retrieve file properties and remember a file or folder so that your app can easily access it again.

We'll look at code to write and read text from a file and how to access the app's local, roaming, and temporary folders.

What do you need to know

Here are the main types you need to know about to read or write text from/to a file:

- [Windows.Storage.StorageFile](#) represents a file. This class has properties that provide information about the file, and methods for creating, opening, copying, deleting, and renaming files. You may be used to dealing with string paths. There are some Windows Runtime APIs that take a string path, but more often you will use a **StorageFile** to represent a file because some files you work with in UWP may not have a path, or may have an unwieldy path. Use [StorageFile.GetFileFromPathAsync\(\)](#) to convert a string path to a **StorageFile**.
- The [FileIO](#) class provides an easy way to read and write text. This class can also read/write an array of bytes, or the contents of a buffer. This class is very similar to the [PathIO](#) class. The main difference is that instead of taking a string path, as **PathIO** does, it takes a **StorageFile**.
- [Windows.Storage.StorageFolder](#) represents a folder (directory). This class has methods for creating files, querying the contents of a folder, creating, renaming, and deleting folders, and properties that provide information about a folder.

Common ways to get a **StorageFolder** include:

- [Windows.Storage.Pickers.FolderPicker](#) which allows the user to navigate to the folder they want to use.

- [Windows.Storage.ApplicationData.Current](#) which provides the **StorageFolder** specific to one of folders local to the app like the local, roaming, and temporary folder.
- [Windows.Storage.KnownFolders](#) which provides the **StorageFolder** for known libraries such as the Music or Picture libraries.

Write text to a file

For this introduction, we will focus on a simple scenario: reading and writing text. Let's start by looking at some code that uses the **FileIO** class to write text to a file.

C#

```
Windows.Storage.StorageFolder storageFolder =  
Windows.Storage.ApplicationData.Current.LocalFolder;  
Windows.Storage.StorageFile file = await  
storageFolder.CreateFileAsync("test.txt",  
    Windows.Storage.CreationCollisionOption.OpenIfExists);  
  
await Windows.Storage.FileIO.WriteTextAsync(file, "Example of writing a  
string\r\n");  
  
// Append a list of strings, one per line, to the file  
var listOfStrings = new List<string> { "line1", "line2", "line3" };  
await Windows.Storage.FileIO.AppendLinesAsync(file, listOfStrings); // each  
entry in the list is written to the file on its own line.
```

We first identify where the file should be located.

`Windows.Storage.ApplicationData.Current.LocalFolder` provides access to the local data folder, which is created for your app when it is installed. See [Access the file system](#) for details about the folders your app can access.

Then, we use **StorageFolder** to create the file (or open it if it already exists).

The **FileIO** class provides a convenient way to write text to the file.

`FileIO.WriteTextAsync()` replaces the entire contents of the file with the provided text.

`FileIO.AppendLinesAsync()` appends a collection of strings to the file--writing one string per line.

Read text from a file

As with writing a file, reading a file starts with specifying where the file is located. We'll use the same location as in the example above. Then we'll use the **FileIO** class to read its contents.

C#

```
Windows.Storage.StorageFolder storageFolder =  
Windows.Storage.ApplicationData.Current.LocalFolder;  
Windows.Storage.StorageFile file = await  
storageFolder.GetFilesAsync("test.txt");  
  
string text = await Windows.Storage.FileIO.ReadTextAsync(file);
```

You can also read each line of the file into individual strings in a collection with

```
IList<string> contents = await Windows.Storage.FileIO.ReadLinesAsync(sampleFile);
```

Access the file system

In the UWP platform, folder access is restricted to ensure the integrity and privacy of the user's data.

App folders

When a UWP app is installed, several folders are created under `c:\users<user name>\AppData\Local\Packages<app package identifier>\` to store, among other things, the app's local, roaming, and temporary files. The app doesn't need to declare any capabilities to access these folders, and these folders are not accessible by other apps. These folders are also removed when the app is uninstalled.

These are some of the app folders you will commonly use:

- **LocalState:** For data local to the current device. When the device is backed up, data in this directory is saved in a backup image in OneDrive. If the user resets or replaces the device, the data will be restored. Access this folder with `Windows.Storage.ApplicationData.Current.LocalFolder`. Save local data that you don't want backed up to OneDrive in the **LocalCacheFolder**, which you can access with `Windows.Storage.ApplicationData.Current.LocalCacheFolder`.
- **RoamingState:** Data stored here no longer roam (as of Windows 11), but the folder is still available. Access the roaming folder with `Windows.Storage.ApplicationData.Current.RoamingFolder`. The recommended replacement is [Azure App Service](#). Azure App Service is widely supported, well documented, reliable, and supports cross-platform/cross-ecosystem scenarios such as iOS, Android and web.

- **TempState:** For data that may be deleted when the app isn't running. Access this folder with `Windows.Storage.ApplicationData.Current.TemporaryFolder`.

Access the rest of the file system

A UWP app must declare its intent to access a specific user library by adding the corresponding capability to its manifest. The user is then prompted when the app is installed to verify that they authorize accessing the specified library. If not, the app is not installed. There are capabilities to access the pictures, videos, and music libraries. See [App capability declaration](#) for a full list. To get a **StorageFolder** for these libraries, use the [Windows.Storage.KnownFolders](#) class.

Documents library

Although there is a capability to access the user's documents library, that capability is restricted which means that an app declaring it will be rejected by the Microsoft Store unless you follow a process to get special approval. It is not intended for general use. Instead, use the file or folder pickers (see [Open files and folders with a picker](#) and [Save a file with a picker](#)) which allow the user to navigate to the folder or file. When the user navigates to a folder or file, they have implicitly given permission for the app to access it and the system allows access.

General access

Alternatively, your app can declare the restricted [broadFileSystem](#) capability in its manifest, which also requires Microsoft Store approval. Then the app can access any file that the user has access to without requiring the intervention of a file or folder picker.

For a comprehensive list of the locations that apps can access, see [File access permissions](#).

Useful APIs and docs

Here is a quick summary of APIs, and other useful documentation, to help get you started with files and folders.

Useful APIs

API	Description
-----	-------------

API	Description
Windows.Storage.StorageFile	Provides information about the file, and methods for creating, opening, copying, deleting, and renaming files.
Windows.Storage.StorageFolder	Provides information about the folder, methods for creating files, and methods for creating, renaming, and deleting folders.
FileIO	Provides an easy way to read and write text. This class can also read/write an array of bytes or the contents of a buffer.
PathIO	Provides an easy way to read/write text from/to a file given a string path for the file. This class can also read/write an array of bytes or the contents of a buffer.
DataReader & DataWriter	Read and write buffers, bytes, integers, GUIDs, TimeSpans, and more, from/to a stream.
Windows.Storage.ApplicationData.Current	Provides access to the folders created for the app such as the local folder, roaming folder, and temporary files folder.
Windows.Storage.Pickers.FolderPicker	Lets the user choose a folder and returns a StorageFolder for it. This is how you get access to locations that the app can't access by default.
Windows.Storage.Pickers.FileOpenPicker	Lets the user choose a file to open and returns a StorageFile for it. This is how you get access to a file that the app can't access by default.
Windows.Storage.Pickers.FileSavePicker	Lets the user choose the file name, extension, and storage location for a file. Returns a StorageFile . This is how you save a file to a location that the app can't access by default.
Windows.Storage.Streams namespace	Covers reading and writing streams. In particular, look at the DataReader and DataWriter classes which read and write buffers, bytes, integers, GUIDs, TimeSpans, and more.

Useful docs

Topic	Description
Windows.Storage Namespace	API reference docs.

Topic	Description
Files, folders, and libraries	Conceptual docs.
Create, write, and read a file	Covers creating, reading, and writing text, binary data, and streams.
Getting started storing app data locally ↗	In addition to covering best practices for saving local data, covers the purpose of the LocalSettings and LocalCache folder.
Store and retrieve settings and other app data	Provides an overview of the various app data stores such as the local, roaming, and temporary folders.
File access permissions	Information about which file system locations your app can access.
Open files and folders with a picker	Shows how to access files and folders by letting the user decide via a picker UI.
Windows.Storage.Streams	Types used to read and write streams.
Files and folders in the Music, Pictures, and Videos libraries	Covers how to remove folders from libraries, get the list of folders in a library, and discover stored photos, music, and videos.

Useful code samples

Code sample	Description
Application data sample	Shows how to store and retrieve data that is specific to each user by using the application data APIs.
File access sample ↗	Shows how to create, read, write, copy and delete a file.
File picker sample	Shows how to access files and folders by letting the user choose them with UI, and how to save a file so that the user can specify the name, file type, and location of a file to save.
JSON sample ↗	Shows how to encode and decode JavaScript Object Notation (JSON) objects, arrays, strings, numbers and booleans using the Windows.Data.Json namespace .
Additional code samples	Choose Files, folder, and libraries in the category drop-down list.

Create your first app

Article • 08/21/2024

Welcome to the [UWP platform](#)! These tutorials will help you create your first UWP app in the language of your choice.

ⓘ Note

If you are starting to develop Windows apps, we recommend you consider using the [Windows App SDK](#), and [WinUI](#) rather than UWP. Although still supported, UWP is not under active development. Please see [Start developing Windows apps](#) for more information.

To get started, choose your favorite language.

C# and XAML

Use your .NET, WPF, or Silverlight skills to build apps using C# with Extensible Application Markup Language (XAML).

- [Create a "Hello, World!" app using XAML with C#](#)

If you want to learn the basics, or just refresh your memory, try reading these:

- [C# Fundamentals for Absolute Beginners](#)
- [VB Fundamentals for Absolute Beginners](#)
- [Develop Windows 10 applications](#)

C++/WinRT and XAML

Leverage your programming expertise in standard C++ to build apps using C++/WinRT with Extensible Application Markup Language (XAML).

- [Create a "Hello, World!" app using C++/WinRT](#)

Learn more about C++/WinRT.

- [C++/WinRT](#)

Visual C++ component extensions (C++/CX) and XAML

Take advantage of your C++ programming expertise to build apps using Visual C++ component extensions (C++/CX) with Extensible Application Markup Language (XAML).

- [Create a "Hello, World!" app with C++/CX](#)

Learn more about C++ here.

- [C++ for UWP apps](#)
- [C++ for Desktop apps](#)

Next steps

When you're ready to try something a little more fun than "Hello, World!", try out these tutorials.

- [Tutorial: Navigate between two pages](#)
- [Microsoft Learn training resources for Windows apps](#)

See Also

- [Start coding](#)
- [What's a UWP app?](#)
- [Get set up](#)
- [Sign up for Windows account](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Create a "Hello, World!" UWP app (XAML)

Article • 11/01/2022

This tutorial teaches you how to use XAML and C# to create a simple "Hello, World!" app for the Universal Windows Platform (UWP) on Windows. With a single project in Microsoft Visual Studio, you can build an app that runs on all versions of Windows 10 and Windows 11.

Here you'll learn how to:

- Create a new **Visual Studio** project that targets **Windows 10** and the **UWP**.
- Write XAML to change the UI on your start page.
- Run the project on the local desktop in Visual Studio.
- Use a `SpeechSynthesizer` to make the app talk when you press a button.

Before you start..

- [What's a Universal Windows app?](#)
- [Download Visual Studio \(and Windows\)](#). If you need a hand, learn how to [get set up](#).
- We also assume you're using the default window layout in Visual Studio. If you change the default layout, you can reset it in the **Window** menu by using the **Reset Window Layout** command.

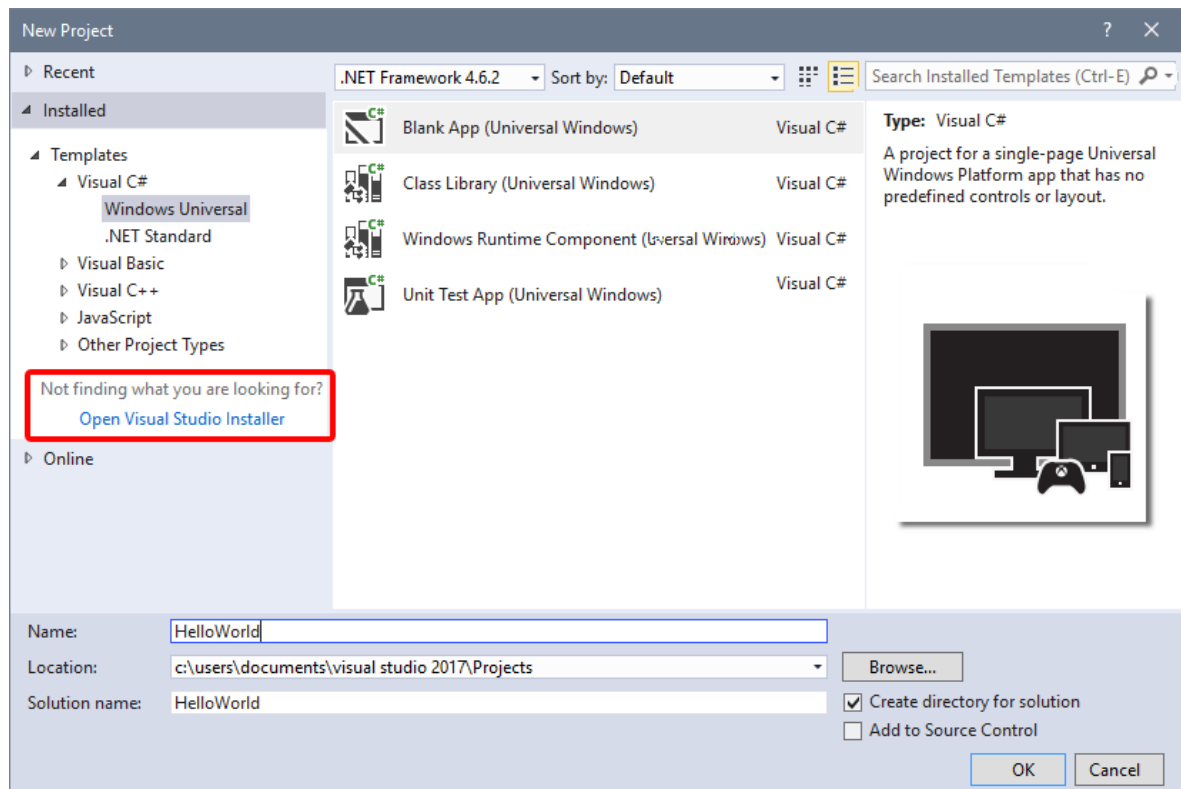
ⓘ Note

This tutorial is using Visual Studio Community 2017. If you are using a different version of Visual Studio, it may look a little different for you.

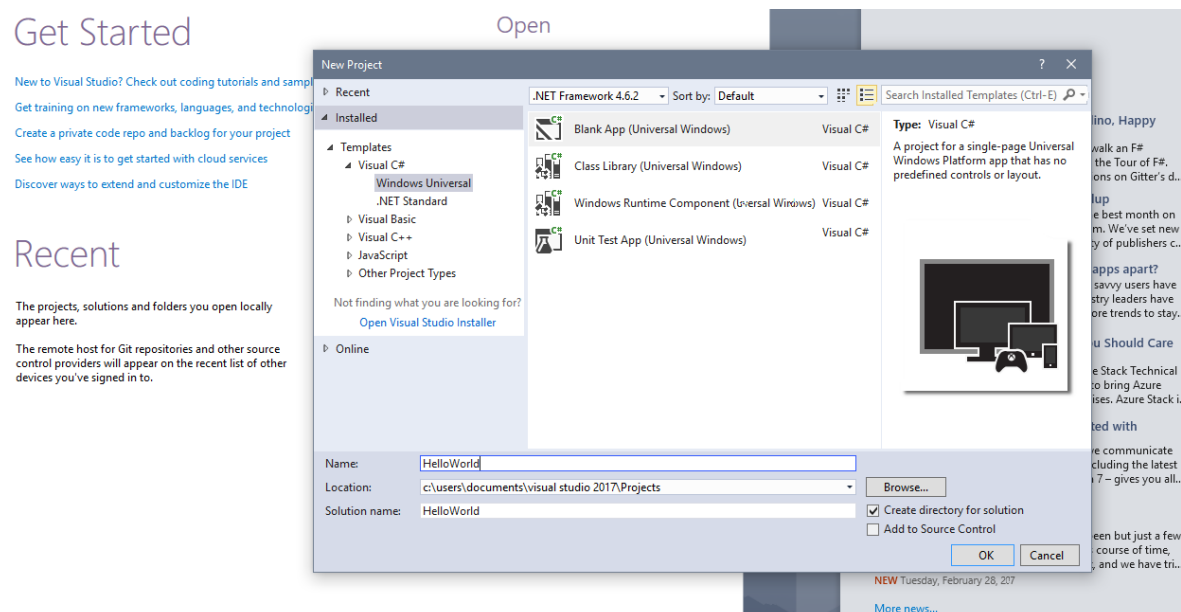
Step 1: Create a new project in Visual Studio

1. Launch Visual Studio.
2. From the **File** menu, select **New > Project** to open the *New Project* dialog.
3. From the list of templates on the left, choose **Installed > Visual C# > Windows Universal** to see the list of UWP project templates.

(If you don't see any Universal templates, you might be missing the components for creating UWP apps. You can repeat the installation process and add UWP support by clicking **Open Visual Studio installer** on the *New Project* dialog. See [Get set up.](#))



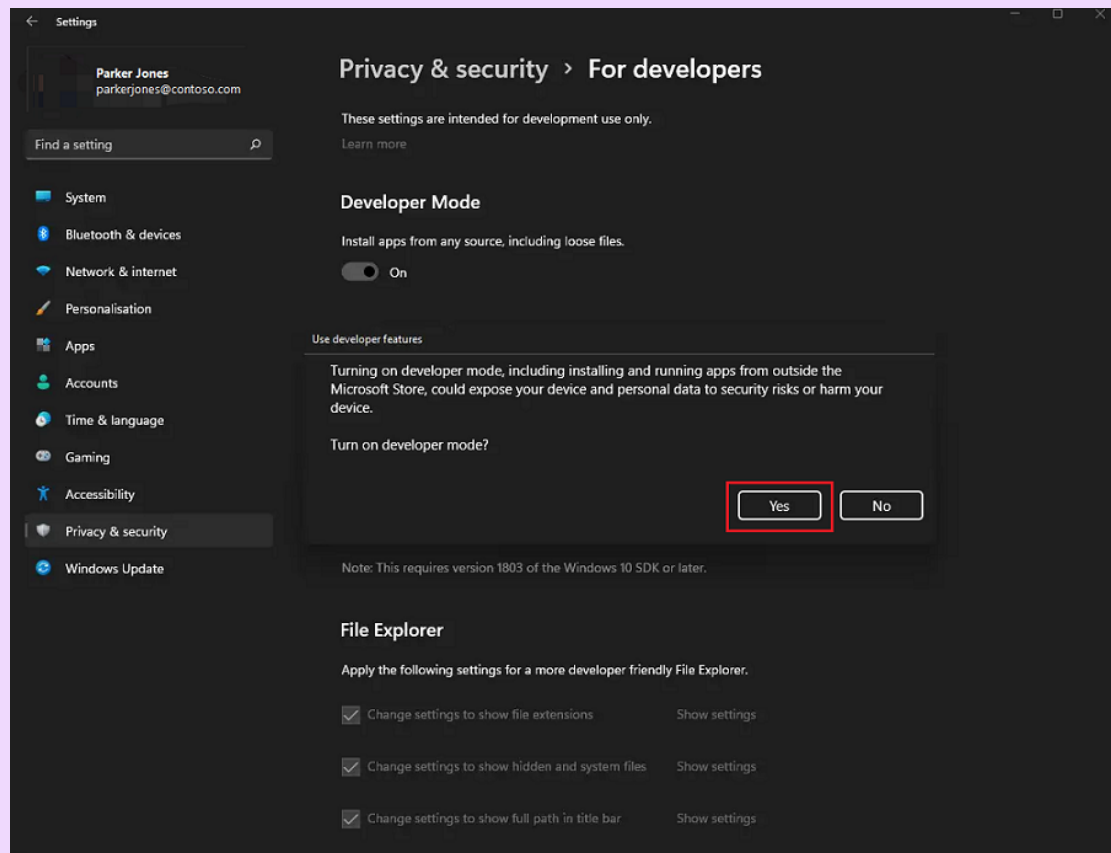
4. Choose the **Blank App (Universal Windows)** template, and enter "HelloWorld" as the **Name**. Select **OK**.



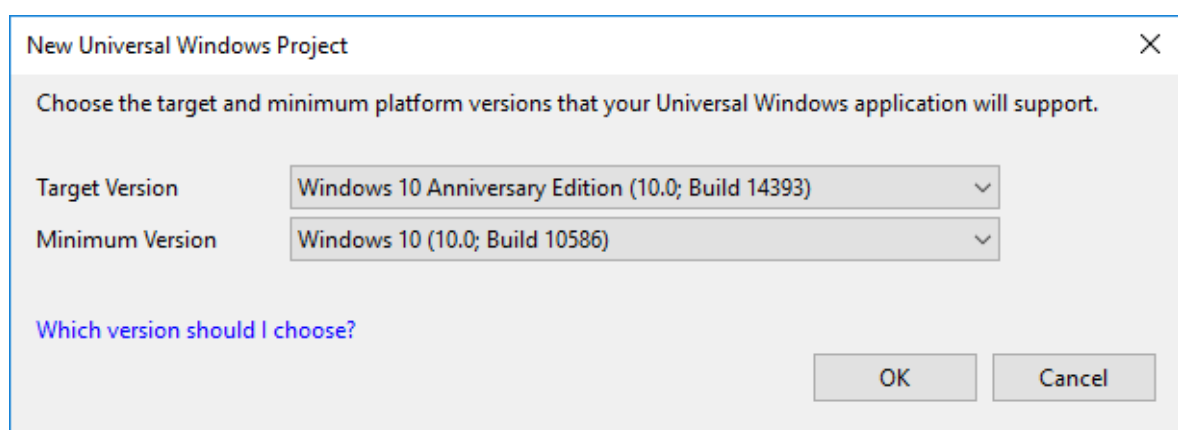
ⓘ Note

If this is the first time you have used Visual Studio, you might see a Settings dialog asking you to enable **Developer mode**. Developer mode is a special

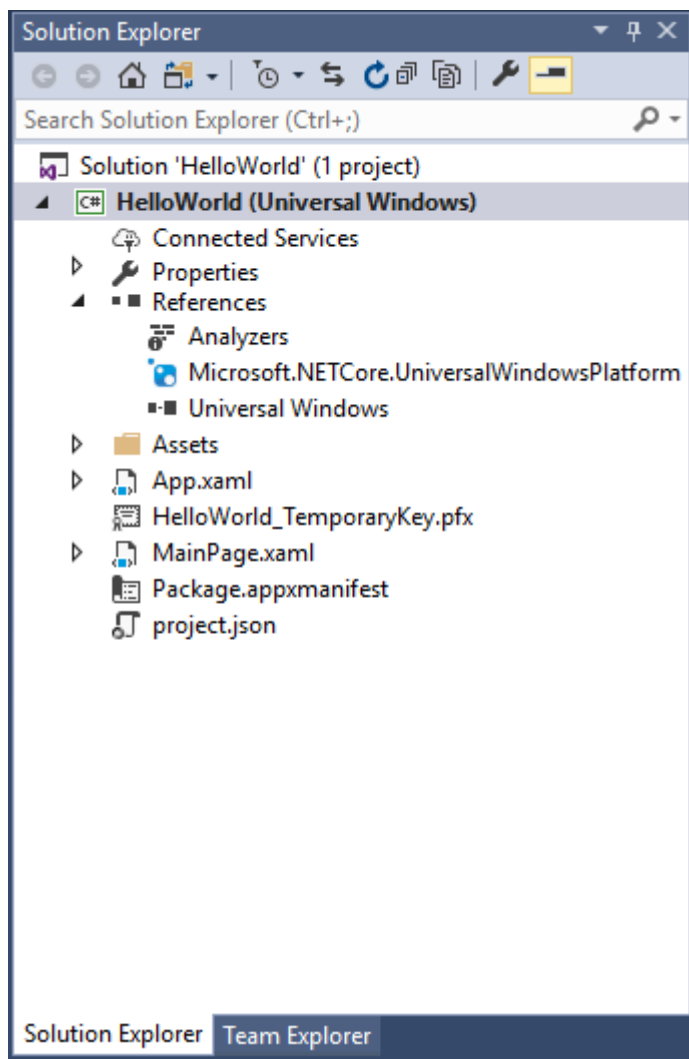
setting that enables certain features, such as permission to run apps directly, rather than only from the Store. For more information, please read [Enable your device for development](#). To continue with this guide, select **Developer mode**, click **Yes**, and close the dialog.



5. The target version/minimum version dialog appears. The default settings are fine for this tutorial, so select **OK** to create the project.



6. When your new project opens, its files are displayed in the **Solution Explorer** pane on the right. You may need to choose the **Solution Explorer** tab instead of the **Properties** tab to see your files.



Although the **Blank App (Universal Window)** is a minimal template, it still contains a lot of files. These files are essential to all UWP apps using C#. Every project that you create in Visual Studio contains them.

What's in the files?

To view and edit a file in your project, double-click the file in the **Solution Explorer**. Expand a XAML file just like a folder to see its associated code file. XAML files open in a split view that shows both the design surface and the XAML editor.

ⓘ Note

What is XAML? Extensible Application Markup Language (XAML) is the language used to define your app's user interface. It can be entered manually, or created using the Visual Studio design tools. A .xaml file has a .xaml.cs code-behind file which contains the logic. Together, the XAML and code-behind make a complete class. For more information, see [XAML overview](#).

App.xaml and App.xaml.cs

- App.xaml is where you declare resources that are used across the app.
- App.xaml.cs is the code-behind file for App.xaml. Like all code-behind pages, it contains a constructor that calls the `InitializeComponent` method. You don't write the `InitializeComponent` method. It's generated by Visual Studio, and its main purpose is to initialize the elements declared in the XAML file.
- App.xaml.cs is the entry point for your app.
- App.xaml.cs also contains methods to handle [activation](#) and [suspension](#) of the app.

MainPage.xaml

- MainPage.xaml is where you define the UI for your app. You can add elements directly using XAML markup, or you can use the design tools provided by Visual Studio.
- MainPage.xaml.cs is the code-behind page for MainPage.xaml. It's where you add your app logic and event handlers.
- Together these two files define a new class called `MainPage`, which inherits from [Page](#), in the `HelloWorld` namespace.

Package.appxmanifest

- A manifest file that describes your app: its name, description, tile, start page, etc.
- Includes a list of dependencies, resources and files that your app contains.

A set of logo images

- Assets/Square150x150Logo.scale-200.png and Wide310x150Logo.scale-200.png represent your app (either Medium or Wide size) in the start menu.
- Assets/Square44x44Logo.png represents your app in the app list of the start menu, task bar and task manager.
- Assets/StoreLogo.png represents your app in the Microsoft Store.
- Assets/SplashScreen.scale-200.png is the splash screen that appears when your app starts.
- Assets/LockScreenLogo.scale-200.png can be used to represent the app on the lock screen, when the system is locked.

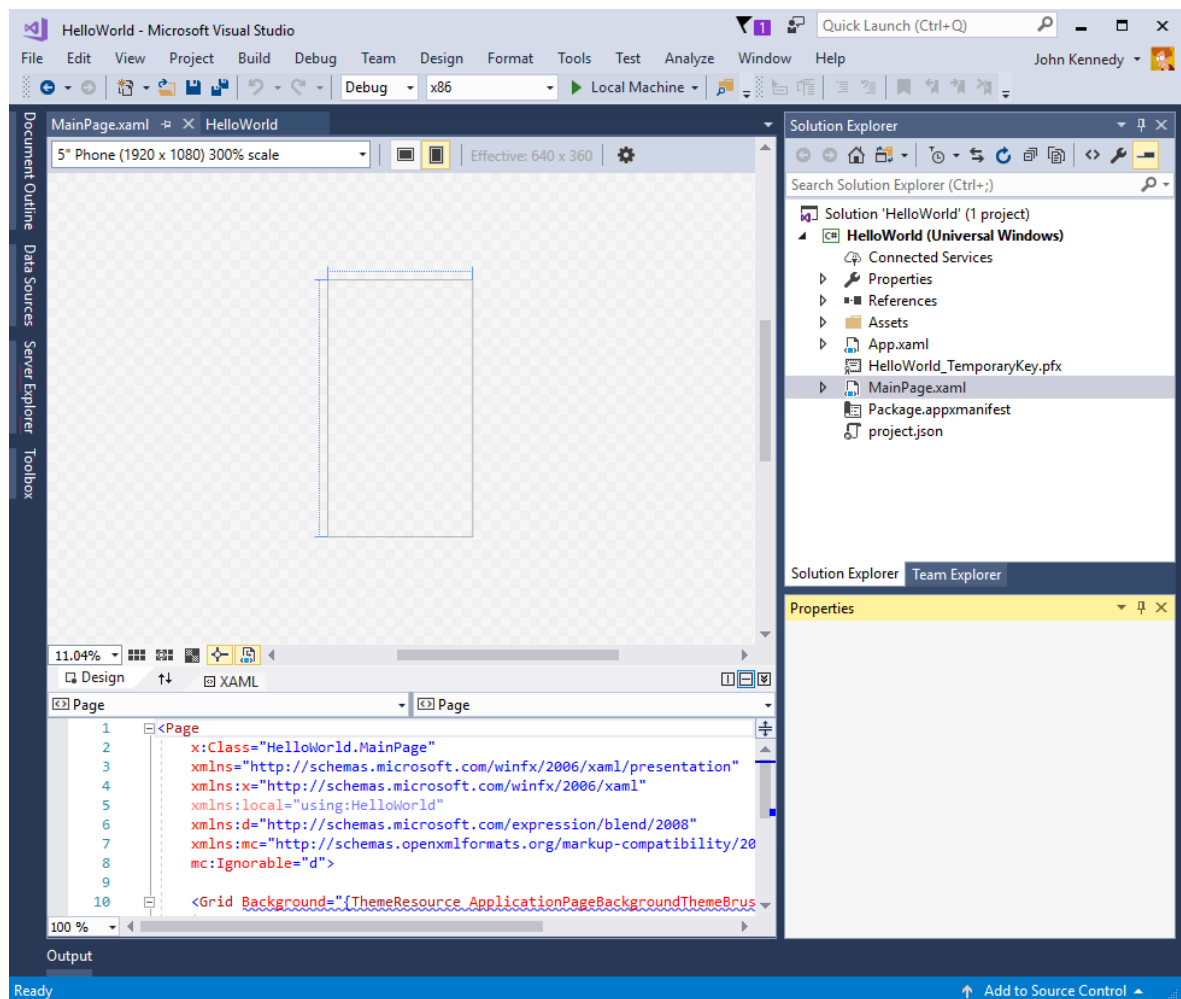
Step 2: Add a button

Using the designer view

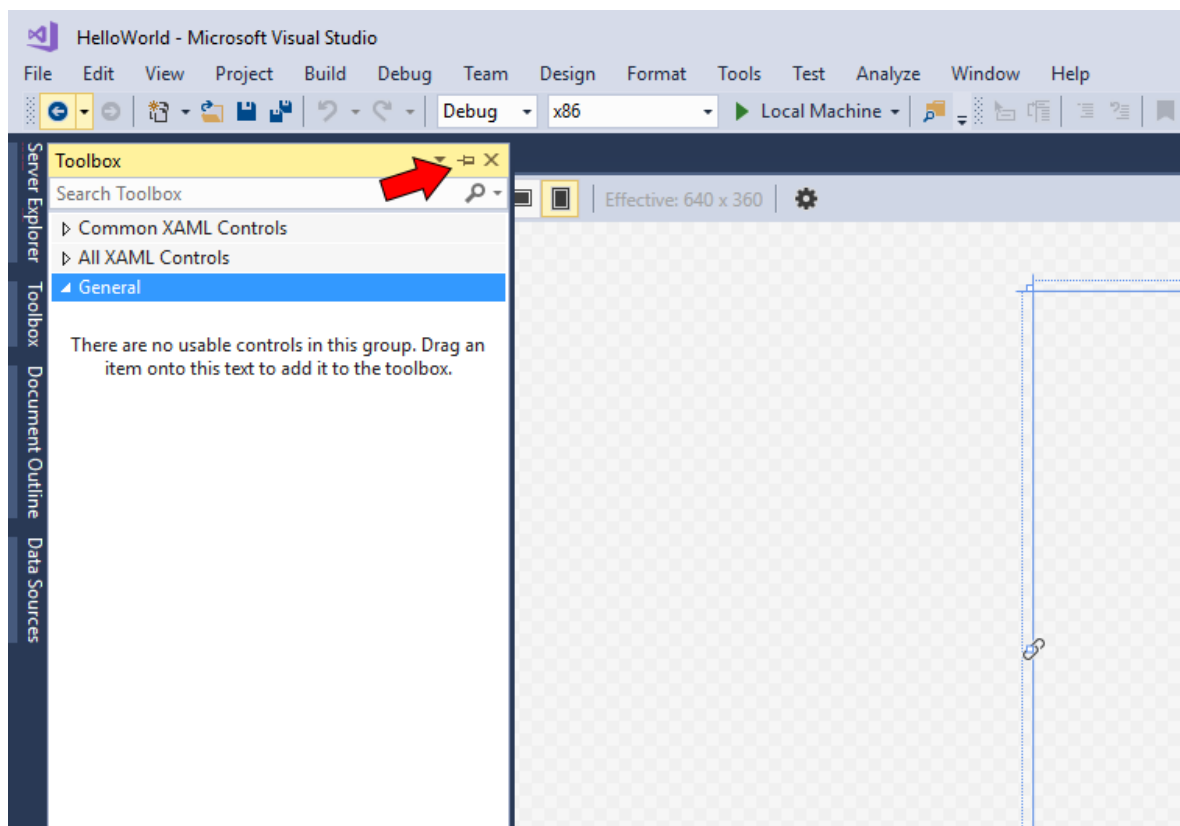
Let's add a button to our page. In this tutorial, you work with just a few of the files listed previously: App.xaml, MainPage.xaml, and MainPage.xaml.cs.

1. Double-click on **MainPage.xaml** to open it in the Design view.

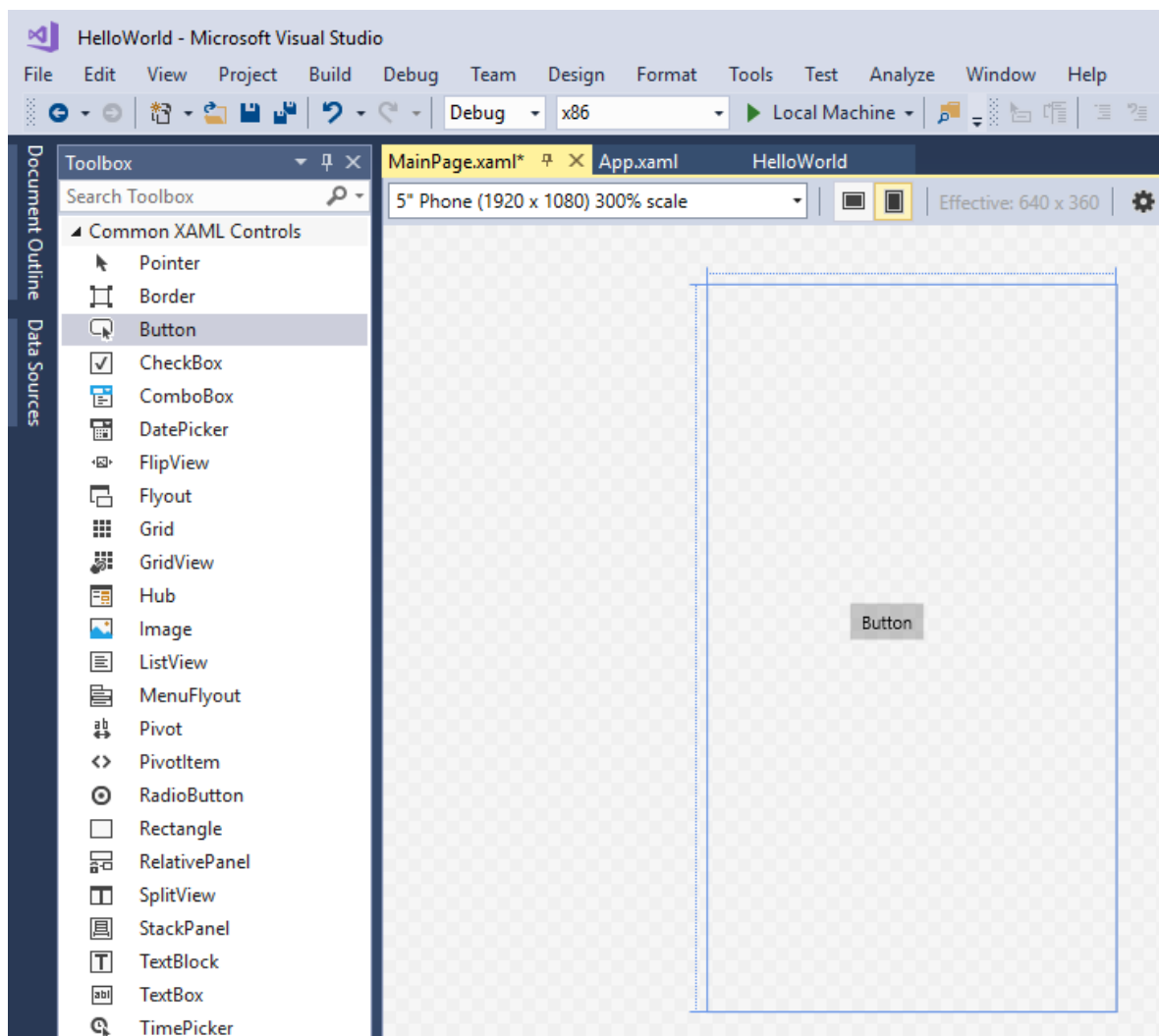
You'll notice there is a graphical view on the top part of the screen, and the XAML code view underneath. You can make changes to either, but for now we'll use the graphical view.



2. Click on the vertical **Toolbox** tab on the left to open the list of UI controls. (You can click the pin icon in its title bar to keep it visible.)



3. Expand **Common XAML Controls**, and drag the **Button** out to the middle of the design canvas.



If you look at the XAML code window, you'll see that the Button has been added there too:

XAML

```
<Button x:Name="button" Content="Button" HorizontalAlignment="Left" Margin =  
"152,293,0,0" VerticalAlignment="Top"/>
```

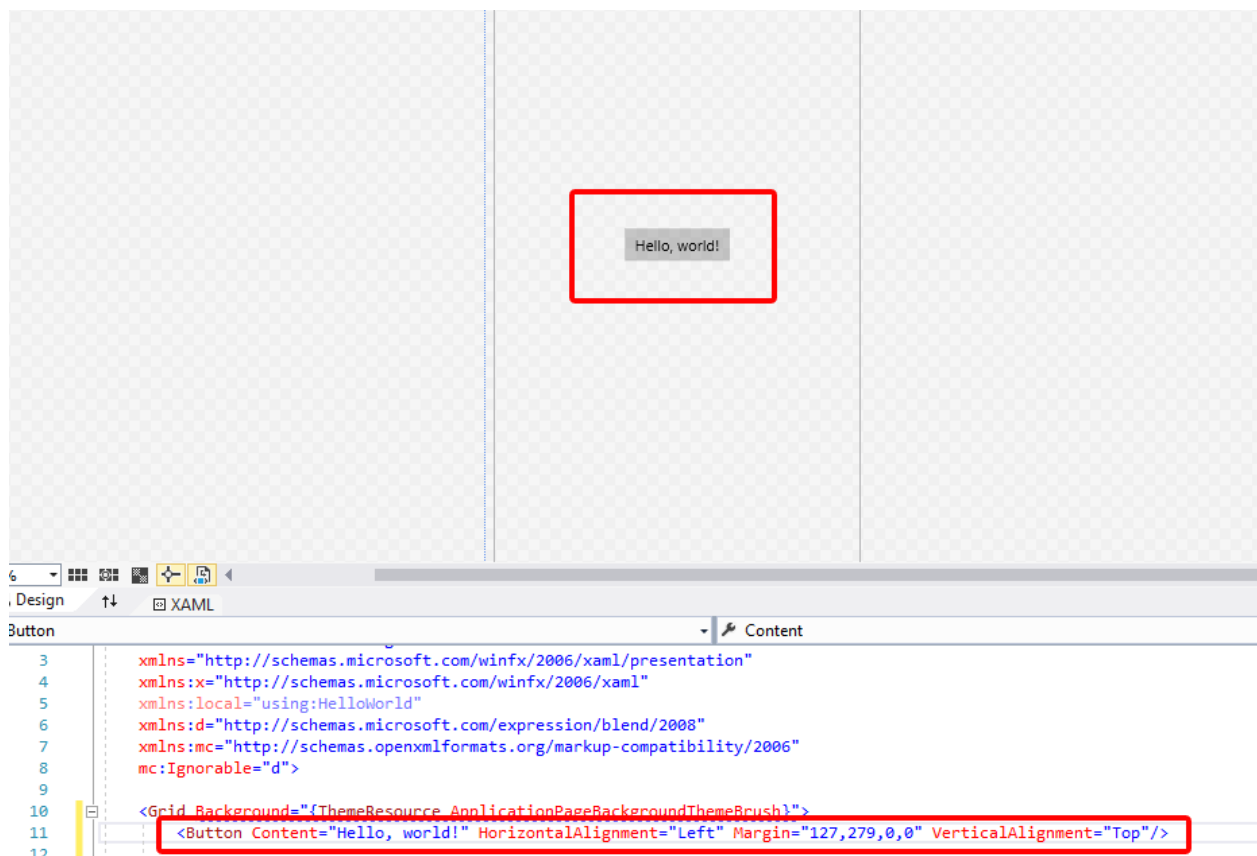
4. Change the button's text.

Click in the XAML code view, and change the Content from "Button" to "Hello, world!".

XAML

```
<Button x:Name="button" Content="Hello, world!" HorizontalAlignment="Left"  
Margin = "152,293,0,0" VerticalAlignment="Top"/>
```

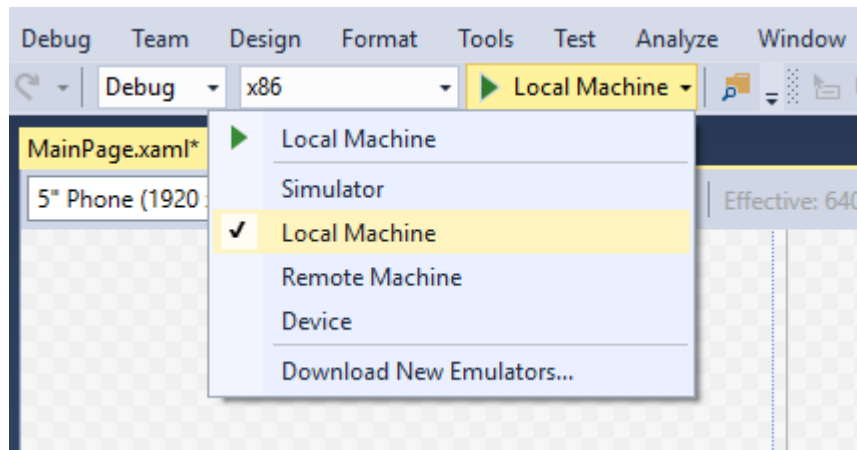
Notice how the button displayed in the design canvas updates to display the new text.



Step 3: Start the app

At this point, you've created a very simple app. This is a good time to build, deploy, and launch your app and see what it looks like. You can debug your app on the local

machine, in a simulator or emulator, or on a remote device. Here's the target device menu in Visual Studio.

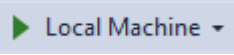


Start the app on a Desktop device

By default, the app runs on the local machine. The target device menu provides several options for debugging your app on devices from the desktop device family.

- Simulator
- Local Machine
- Remote Machine

To start debugging on the local machine

1. In the target device menu () on the **Standard** toolbar, make sure that **Local Machine** is selected. (It's the default selection.)
2. Click the **Start Debugging** button () on the toolbar.

–or–

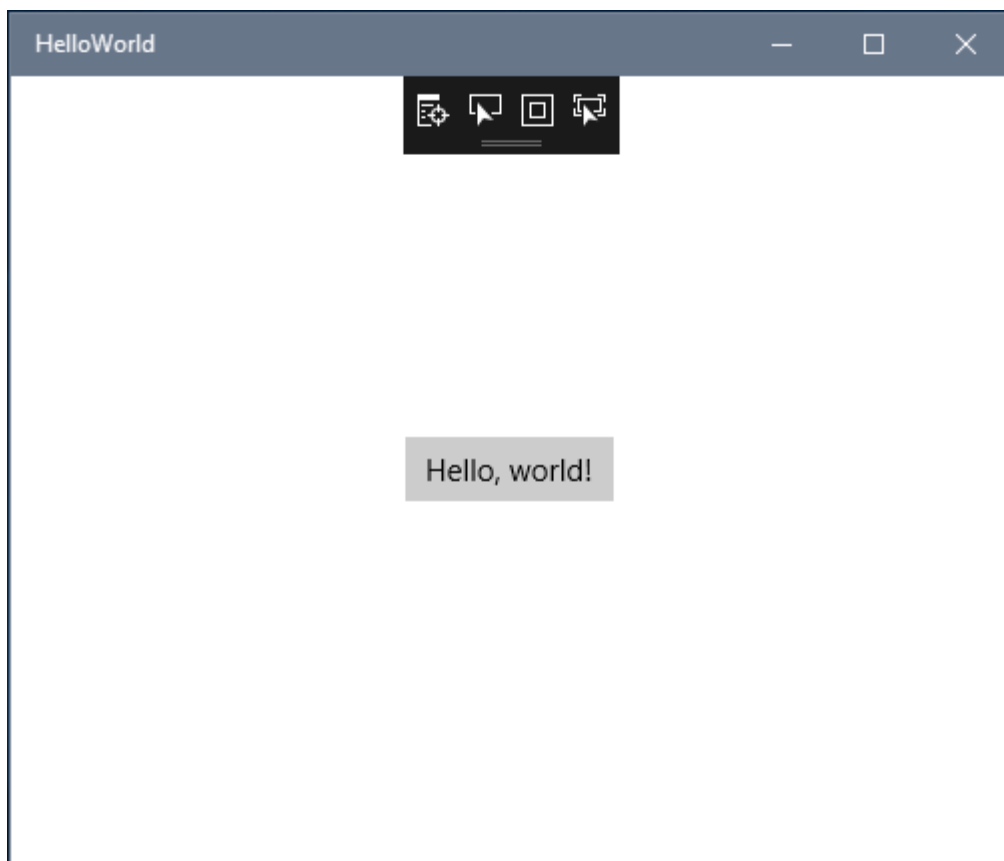
From the **Debug** menu, click **Start Debugging**.

–or–

Press F5.

The app opens in a window, and a default splash screen appears first. The splash screen is defined by an image (SplashScreen.png) and a background color (specified in your app's manifest file).

The splash screen disappears, and then your app appears. It looks like this.



Press the Windows key to open the **Start** menu, then show all apps. Notice that deploying the app locally adds its tile to the **Start** menu. To run the app again later (not in debugging mode), tap or click its tile in the **Start** menu.

It doesn't do much—yet—but congratulations, you've built your first UWP app!

To stop debugging

Click the **Stop Debugging** button () in the toolbar.

—or—

From the **Debug** menu, click **Stop debugging**.

—or—

Close the app window.

Step 4: Event handlers

An "event handler" sounds complicated, but it's just another name for the code that is called when an event happens (such as the user clicking on your button).

1. Stop the app from running, if you haven't already.

2. Double-click on the button control on the design canvas to make Visual Studio create an event handler for your button.

You can of course, create all the code manually too. Or you can click on the button to select it, and look in the **Properties** pane on the lower right. If you switch to **Events** (the little lightning bolt) you can add the name of your event handler.

3. Edit the event handler code in *MainPage.xaml.cs*, the code-behind page. This is where things get interesting. The default event handler looks like this:

```
C#

private void Button_Click(object sender, RoutedEventArgs e)
{

}

}
```

Let's change it, so it looks like this:

```
C#

private async void Button_Click(object sender, RoutedEventArgs e)
{
    MediaElement mediaElement = new MediaElement();
    var synth = new Windows.Media.SpeechSynthesis.SpeechSynthesizer();
    Windows.Media.SpeechSynthesis.SpeechSynthesisStream stream = await
synth.SynthesizeTextToStreamAsync("Hello, World!");
    mediaElement.SetSource(stream, stream.ContentType);
    mediaElement.Play();
}

}
```

Make sure the method signature now includes the **async** keyword, or you'll get an error when you try to run the app.

What did we just do?

This code uses some Windows APIs to create a speech synthesis object, and then gives it some text to say. (For more information on using `SpeechSynthesis`, see the [SpeechSynthesis namespace docs](#).)

When you run the app and click on the button, your computer (or phone) will literally say "Hello, World!".

Summary

Congratulations, you've created your first app for Windows and the UWP!

To learn how to use XAML for laying out the controls your app will use, try the [grid tutorial](#), or jump straight to [next steps](#)?

See Also

- [Your first app](#)
- [Publishing your UWP app.](#)
- [How-to articles on developing UWP apps](#)
- [Code Samples for UWP developers](#)
- [What's a Universal Windows app?](#)
- [Sign up for Windows account](#)

Create a "Hello, World!" UWP app using C++/WinRT

Article • 10/20/2022

This topic walks you through creating a Windows Universal Windows Platform (UWP) "Hello, World!" app using C++/WinRT. The app's user interface (UI) is defined using Extensible Application Markup Language (XAML).

C++/WinRT is an entirely standard modern C++17 language projection for Windows Runtime (WinRT) APIs. For more info, and more walkthroughs and code examples, see the [C++/WinRT](#) documentation. A good topic to begin with is [Get started with C++/WinRT](#).

Set up Visual Studio for C++/WinRT

For info about setting up Visual Studio for C++/WinRT development—including installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support)—see [Visual Studio support for C++/WinRT](#).

To download Visual Studio, see [Downloads](#).

For an introduction to XAML, see [XAML overview](#)

Create a Blank App (HelloWorldCppWinRT)

Our first app is a "Hello, World!" app that demonstrates some basic features of interactivity, layout, and styles.

Begin by creating a new project in Microsoft Visual Studio. Create a **Blank App (C++/WinRT)** project, and name it *HelloWorldCppWinRT*. Make sure that **Place solution and project in the same directory** is unchecked. Target the latest generally-available (that is, not preview) version of the Windows SDK.

In a later section of this topic, you'll be directed to build your project (but don't build until then).

About the project files

Typically, in the project folder, each `.xaml` (XAML markup) file has a corresponding `.idl`, `.h`, and `.cpp` file. Together, those files compile into a XAML page type.

You can modify a XAML markup file to create UI elements, and you can bind those elements to data sources (a task known as [data binding](#)). You modify the `.h`, and `.cpp` files (and sometimes the `.idl` file) to add custom logic for your XAML page—event handlers, for example.

Let's look at some of the project files.

- `App.idl`, `App.xaml`, `App.h`, and `App.cpp`. These files represent your app's specialization of the `Windows::UI::Xaml::Application` class, which includes your app's entry point. `App.xaml` doesn't contain any page-specific markup, but you can add user interface element styles there, as well as any other elements that you want to be accessible from all pages. The `.h` and `.cpp` files contain handlers for various application lifecycle events. Typically, you add custom code there to initialize your app when it starts, and to perform cleanup when it's either suspended or terminated.
- `MainPage.idl`, `MainPage.xaml`, `MainPage.h`, and `MainPage.cpp`. Contain the XAML markup, and implementation, for the default main (startup) page type in an app, which is the **MainPage** runtime class. **MainPage** has no navigation support, but it provides some default UI, and an event handler, to get you started.
- `pch.h` and `pch.cpp`. These files represent your project's precompiled header file. In `pch.h`, include any header files that don't change often, and then include `pch.h` in other files in the project.

A first look at the code

Runtime classes

As you may know, all of the classes in a Universal Windows Platform (UWP) app written in C# are Windows Runtime types. But when you author a type in a C++/WinRT application, you can choose whether that type is a Windows Runtime type, or a regular C++ class/struct/enumeration.

Any XAML page type in your project needs to be a Windows Runtime type. So **MainPage** is a Windows Runtime type. Specifically, it's a *runtime class*. Any type that's consumed by a XAML page also needs to be a Windows Runtime type. When you're writing a [Windows Runtime component](#), and you want to author a type that can be consumed from another app, then you'll author a Windows Runtime type. In other

cases, your type can be a regular C++ type. Generally speaking, a Windows Runtime type can be consumed using any Windows Runtime language.

One good indication that a type is a Windows Runtime type is that it's defined in [Microsoft Interface Definition Language \(IDL\)](#) inside an Interface Definition Language (`.idl`) file. Let's take **MainPage** as an example.

idl

```
// MainPage.idl
namespace HelloWorldCppWinRT
{
    [default_interface]
    runtimeclass MainPage : Windows.UI.Xaml.Controls.Page
    {
        MainPage();
        Int32 MyProperty;
    }
}
```

And here's the basic structure of the implementation of the **MainPage** runtime class, and its activation factory, as seen in `MainPage.h`.

C++/WinRT

```
// MainPage.h
...
namespace winrt::HelloWorldCppWinRT::implementation
{
    struct MainPage : MainPageT<MainPage>
    {
        MainPage();

        int32_t MyProperty();
        void MyProperty(int32_t value);
        ...
    };
}

namespace winrt::HelloWorldCppWinRT::factory_implementation
{
    struct MainPage : MainPageT<MainPage, implementation::MainPage>
    {
    };
}
```

For more details about whether or not you should author a runtime class for a given type, see the topic [Author APIs with C++/WinRT](#). And for more info about the connection between runtime classes and IDL (`.idl` files), you can read and follow along

with the topic [XAML controls; bind to a C++/WinRT property](#). That topic walks through the process of authoring a new runtime class, the first step of which is to add a new **Midl File (.idl)** item to the project.

Now let's add some functionality to the **HelloWorldCppWinRT** project.

Step 1. Modify your startup page

In **Solution Explorer**, open `MainPage.xaml` so that you can author the controls that form the user interface (UI).

Delete the **StackPanel** that's in there already, as well as its contents. In its place, paste the following XAML.

XAML

```
<StackPanel x:Name="contentPanel" Margin="120,30,0,0">
    <TextBlock HorizontalAlignment="Left" Text="Hello, World!"
FontSize="36"/>
    <TextBlock Text="What's your name?"/>
    <StackPanel x:Name="inputPanel" Orientation="Horizontal"
Margin="0,20,0,20">
        <TextBox x:Name="nameInput" Width="300" HorizontalAlignment="Left"/>
        <Button x:Name="inputButton" Content="Say "Hello""/>
    </StackPanel>
    <TextBlock x:Name="greetingOutput"/>
</StackPanel>
```

This new **StackPanel** has a **TextBlock** that prompts for the user's name, a **TextBox** that accepts the user's name, a **Button**, and another **TextBlock** element.

Since we deleted the **Button** named *myButton*, we'll have to remove the reference to it from the code. So, in `MainPage.cpp`, delete the line of code inside the **MainPage::ClickHandler** function.

At this point, you've created a very basic Universal Windows app. To see what the UWP app looks like, build and run the app.

Hello, World!

What's your name?

In the app, you can type into the text box. But clicking the button doesn't do anything yet.

Step 2. Add an event handler

In `MainPage.xaml`, find the **Button** named `inputButton`, and declare an event handler for its **ButtonBase::Click** event. The markup for the **Button** should now look like this.

XAML

```
<Button x:Name="inputButton" Content="Say 'Hello'"
Click="inputButton_Click"/>
```

Implement the event handler like this.

C++/WinRT

```
// MainPage.h
struct MainPage : MainPageT<MainPage>
{
    ...
    void inputButton_Click(
        winrt::Windows::Foundation::IIinspectable const& sender,
        winrt::Windows::UI::Xaml::RoutedEventArgs const& e);
};

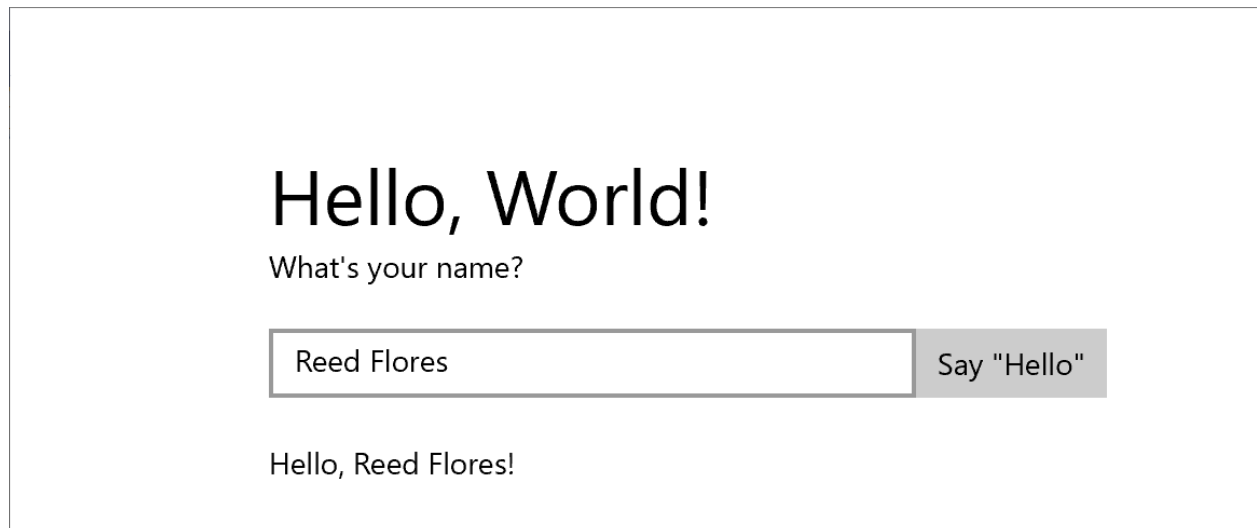
// MainPage.cpp
namespace winrt::HelloWorldCppWinRT::implementation
{
    ...
    void MainPage::inputButton_Click(
        winrt::Windows::Foundation::IIinspectable const& sender,
        winrt::Windows::UI::Xaml::RoutedEventArgs const& e)
    {
        greetingOutput().Text(L"Hello, " + nameInput().Text() + L"!");
    }
}
```

```
}  
}
```

For more info, see [Handle events by using delegates](#).

The implementation retrieves the user's name from the text box, uses it to create a greeting, and displays that in the *greetingOutput* text block.

Build and run the app. Type your name in the text box, and click the button. The app displays a personalized greeting.



Step 3. Style the startup page

Choose a theme

It's easy to customize the look and feel of your app. By default, your app uses resources that have a light-colored style. The system resources also include a dark theme.

To try out the dark theme, edit `App.xaml`, and add a value for [Application::RequestedTheme](#).

XAML

```
<Application  
  ...  
  RequestedTheme="Dark">  
  
</Application>
```

For apps that display mostly images or video, we recommend the dark theme; for apps that contain a lot of text, we recommend the light theme. If you're using a custom color

scheme, then use the theme that goes best with your app's look and feel.

ⓘ Note

A theme is applied when your app starts up. It can't be changed while the app is running.

Use system styles

In this section we'll change the appearance of the text (for example, make the font size larger).

In `MainPage.xaml`, find the "What's your name?" **TextBlock**. Set its **Style** property to a reference to the *BaseTextBlockStyle* system resource key.

XAML

```
<TextBlock Text="What's your name?" Style="{ThemeResource
BaseTextBlockStyle}"/>
```

BaseTextBlockStyle is the key of a resource that's defined in the **ResourceDictionary** in `\Program Files (x86)\Windows Kits\10\DesignTime\CommonConfiguration\Neutral\UAP\<version>\Generic\generic.xaml`. Here are the property values that are set by that style.

XAML

```
<Style x:Key="BaseTextBlockStyle" TargetType="TextBlock">
  <Setter Property="FontFamily" Value="XamlAutoFontFamily" />
  <Setter Property="FontWeight" Value="SemiBold" />
  <Setter Property="FontSize" Value="14" />
  <Setter Property="TextTrimming" Value="None" />
  <Setter Property="TextWrapping" Value="Wrap" />
  <Setter Property="LineStackingStrategy" Value="MaxHeight" />
  <Setter Property="TextLineBounds" Value="Full" />
</Style>
```

Also in `MainPage.xaml`, find the **TextBlock** named `greetingOutput`. Set its **Style** to *BaseTextBlockStyle* also. If you build and run the app now, you'll see that the appearance of both text blocks has changed (for example, the font size is now larger).

Step 4. Have the UI adapt to different window sizes